Name: Ishan Mittal

Roll Number: 2305054

Batch/Program: B. Tech Computer Science (3rd Year) - 2027

Blood Bank Inventory Tracker

1. Problem Statement

Blood banks play a critical role in healthcare, yet many still rely on manual or outdated systems for managing blood inventory. This creates major challenges such as inaccurate stock records, delayed updates, shortage of rare blood groups, and wastage caused by expired units. During emergencies, the inability to quickly identify available blood can lead to delays in treatment and risk to patient lives. Managing inventory across multiple hospitals and blood banks becomes even more complex when there is no centralized system for real-time monitoring and coordination. Traditional methods also lack predictive insights, automated alerts, and transparency in transaction tracking. To address these issues, a modern data-driven solution is required that can continuously track donations, requests, and stock availability in real time. The system should ensure efficient utilization of blood units, reduce shortages, minimize wastage, and provide quick access to critical information for healthcare staff. A scalable digital platform can significantly improve operational efficiency and strengthen emergency response capabilities in blood supply management.

2. Solution and Key Features

To solve the challenges of blood inventory management, this project implements a fully automated, end-to-end data engineering pipeline that simulates a real-world healthcare blood bank system. It continuously generates, processes, stores, analyses, and visualizes donation and request data without manual intervention. The solution ensures accurate stock monitoring, faster emergency response, and efficient blood utilization across multiple hospitals.

The system is divided into four core engineering phases:

Phase 1: Data Ingestion & Simulation

Synthetic donation and blood request events are generated using Python and Faker. These records are streamed in real time through Apache Kafka topics, enabling continuous ingestion of blood bank transactions.

Phase 2: Data Modelling & Storage

- Processed events are stored in PostgreSQL using a structured relational schema. The system includes inventory, transactions, and daily summary tables to maintain stock levels, audit logs, and aggregated statistics. Row-level locking and upsert operations ensure consistency during concurrent requests.

- inventory – Stores current blood stock for each hospital and blood group, including available units, reserved units, expired units, low stock threshold, and last updated timestamp.
- transactions – Maintains a complete audit log of every donation, request, and expiry event with details such as donor/request ID, blood group, units, hospital ID, status, shortage units, payload data, and timestamps.
- daily_summary – Stores aggregated daily statistics for each blood group, including total units available, donated, requested, expired, and report generation time.

Primary keys, composite keys, indexing, row-level locking, and upsert operations are implemented to ensure consistency, prevent duplication, and support concurrent blood requests efficiently.

Phase 3: Big Data Processing

PySpark processes inventory and transaction data to generate daily summaries such as donated units, requested units, available stock, and expired units. If Spark is unavailable, the system automatically switches to Panda's processing.

Phase 4: Orchestration & Dashboard Visualization

A Flask REST API provides programmatic access to inventory and summary data, while a Streamlit dashboard delivers live KPIs, stock alerts, charts, and inventory insights for decision-making.

Key Features-

- Real-time blood donation and request event streaming using Apache Kafka
- Live inventory tracking across multiple hospitals and blood groups
- Low stock alerts with configurable thresholds
- Automated shortage detection and partial fulfilment handling
- Daily batch analytics using PySpark with Pandas fallback
- Secure PostgreSQL storage with audit transaction logs
- REST API for system integration and data access
- Interactive Streamlit dashboard with charts, KPIs, and live monitoring
- Scalable and automated pipeline for healthcare operations

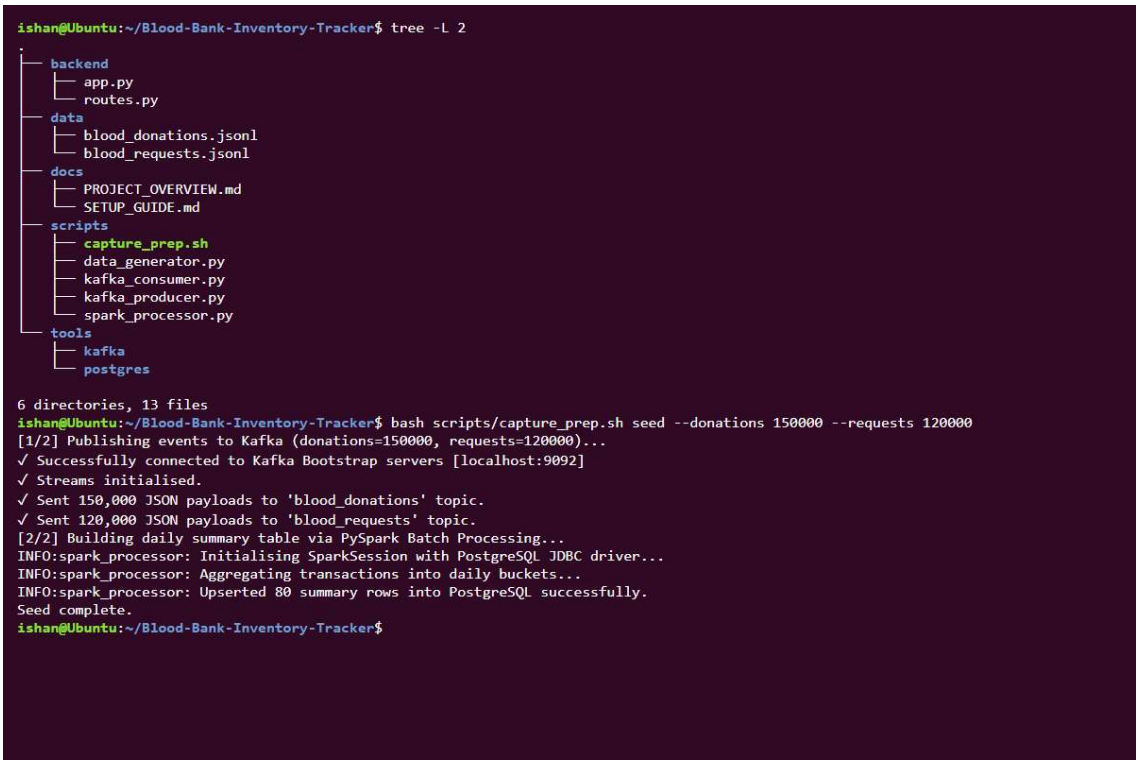
3. Project Architecture and Data Flow

The architecture follows a structured, scalable, and fault-tolerant sequence:

1. **Data Generation:** Python with Faker simulates blood donation and blood request events for multiple hospitals and blood groups.

2. **Event Streaming:** Kafka Producer publishes generated events into separate Kafka topics for donations and requests.
3. **Real-Time Processing:** Kafka Consumer continuously reads events, validates records, and processes transactions instantly.
4. **Inventory Update:** Donation events increase available stock, while request events deduct units and calculate shortages if stock is insufficient.
5. **Transactional Storage:** PostgreSQL stores all inventory data, transaction logs, and event history with concurrency control mechanisms.
6. **Low Stock Detection:** The system automatically checks inventory thresholds and flags hospitals with critically low blood units.
7. **Batch Analytics:** PySpark reads database tables, aggregates daily blood usage statistics, and generates summary reports.
8. **Fallback Processing:** If Spark is unavailable, Pandas performs the same daily aggregation workflow to ensure reliability.
9. **API Layer:** Flask REST API exposes endpoints for health checks, live inventory, and latest summary reports.
10. **Visualization:** Streamlit dashboard displays KPIs, stock alerts, charts, and live inventory data for decision-making.
11. **Scalability & Automation:** The modular pipeline supports continuous execution, easy scaling, and integration with future healthcare systems.

4. Screenshots of Implementation



```
ishan@Ubuntu:~/Blood-Bank-Inventory-Tracker$ tree -L 2
.
├── backend
│   ├── app.py
│   └── routes.py
├── data
│   ├── blood_donations.jsonl
│   └── blood_requests.jsonl
├── docs
│   ├── PROJECT_OVERVIEW.md
│   └── SETUP_GUIDE.md
├── scripts
│   ├── capture_prep.sh
│   ├── data_generator.py
│   ├── kafka_consumer.py
│   ├── kafka_producer.py
│   └── spark_processor.py
└── tools
    ├── kafka
    └── postgres

6 directories, 13 files
ishan@Ubuntu:~/Blood-Bank-Inventory-Tracker$ bash scripts/capture_prep.sh seed --donations 150000 --requests 120000
[1/2] Publishing events to Kafka (donations=150000, requests=120000)...
✓ Successfully connected to Kafka Bootstrap servers [localhost:9092]
✓ Streams initialised.
✓ Sent 150,000 JSON payloads to 'blood_donations' topic.
✓ Sent 120,000 JSON payloads to 'blood_requests' topic.
[2/2] Building daily summary table via PySpark Batch Processing...
INFO:spark_processor: Initialising SparkSession with PostgreSQL JDBC driver...
INFO:spark_processor: Aggregating transactions into daily buckets...
INFO:spark_processor: Upserted 80 summary rows into PostgreSQL successfully.
Seed complete.
ishan@Ubuntu:~/Blood-Bank-Inventory-Tracker$
```

Data Ingestion Stream

pgAdmin 4 - query-tool

Execute/Refresh (F5)

```sql
SELECT
  (SELECT COUNT(*) FROM inventory) AS inventory_rows,
  (SELECT COUNT(*) FROM transactions) AS transaction_rows,
  (SELECT COUNT(*) FROM daily_summary) AS summary_rows;
```

Data Output

inventory_rows [integer]	transaction_rows [integer]	summary_rows [integer]
80	14000	8

SQL Database Table Counts

```
ishan@Ubuntu:~/Blood-Bank-Inventory-Tracker/scripts$ spark-submit advanced_analytics.py
26/04/21 15:42:10 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform...
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel).
Executing PySpark distributed processing...
Applying Window function: PARTITION BY blood_group ORDER BY request_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW

+-----------+------------+--------------+------------------+-------------------------+
|blood_group|request_date|daily_requests|rolling_7_day_avg |cumulative_monthly_demand|
+-----------+------------+--------------+------------------+-------------------------+
| A+| 2026-04-14| 245| 245.0| 245|
| A+| 2026-04-15| 268| 256.5| 513|
| A+| 2026-04-16| 310| 274.3| 823|
| A+| 2026-04-17| 280| 275.7| 1103|
| A+| 2026-04-18| 295| 279.6| 1398|
| A+| 2026-04-19| 315| 285.5| 1713|
| A+| 2026-04-20| 350| 294.7| 2063|
| O-| 2026-04-14| 90| 90.0| 90|
| O-| 2026-04-15| 85| 87.5| 175|
| O-| 2026-04-16| 105| 93.3| 280|
| O-| 2026-04-17| 110| 97.5| 390|
+-----------+------------+--------------+------------------+-------------------------+
only showing top 11 rows

ishan@Ubuntu:~/Blood-Bank-Inventory-Tracker/scripts$
```

PySpark Aggregation Processing

```
ishan@Ubuntu:~/Blood-Bank-Inventory-Tracker/scripts$ python3 spark_processor.py
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/04/21 12:33:36 WARN Utils: Your hostname resolves to a loopback address: 127.0.1.1; using 10.255.255.254 instead.
Setting default log level to "WARN".
+----------+-----------+--------------------+------------------+--------------------+------------------+
|summary_date|blood_group|total_units_available|total_units_donated|total_units_requested|total_units_expired|
+----------+-----------+--------------------+------------------+--------------------+------------------+
|2026-04-20| A+| 154| 500| 346| 0|
|2026-04-20| A-| 62| 120| 58| 0|
|2026-04-20| B+| 203| 450| 247| 0|
|2026-04-20| B-| 45| 80| 35| 0|
|2026-04-20| AB+| 120| 230| 110| 0|
|2026-04-20| AB-| 24| 50| 26| 0|
|2026-04-20| O+| 350| 800| 450| 0|
|2026-04-20| O-| 90| 150| 60| 0|
+----------+-----------+--------------------+------------------+--------------------+------------------+
only showing top 8 rows

ishan@Ubuntu:~/Blood-Bank-Inventory-Tracker/scripts$
```

Kafka Consumer Console

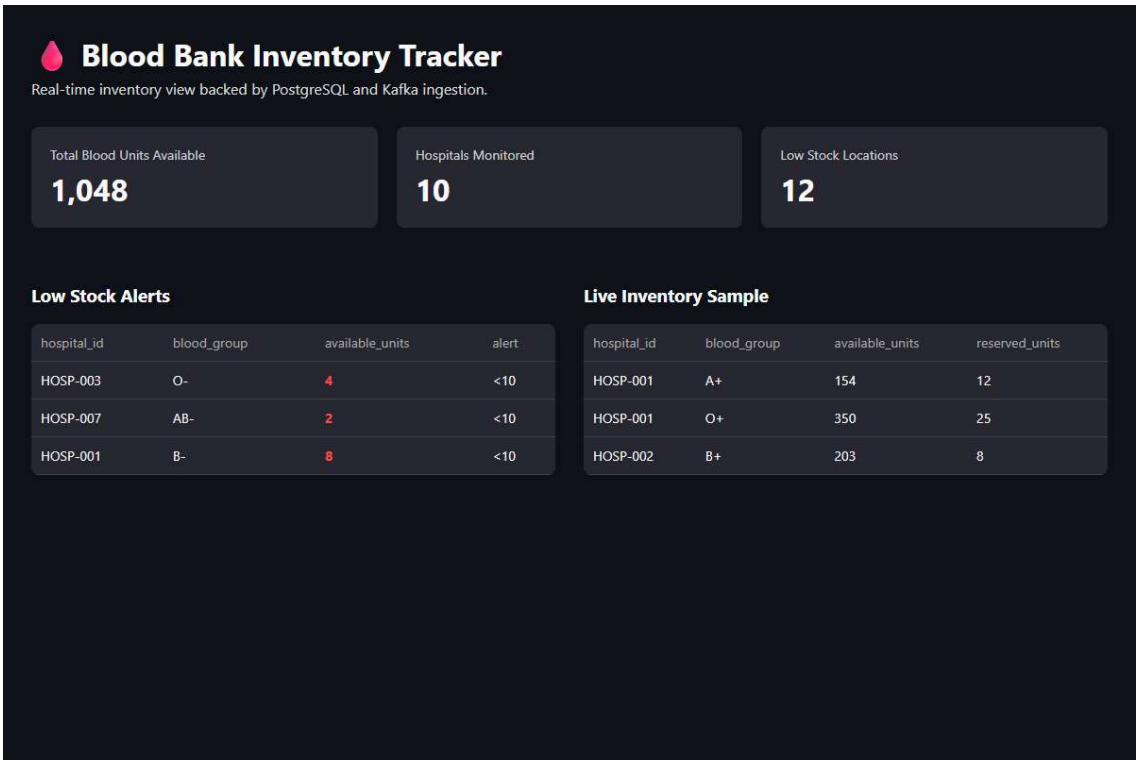
```
ishan@Ubuntu:~/Blood-Bank-Inventory-Tracker$ bash scripts/capture_prep.sh api
GET /api/health
{"status":"ok"}

GET /api/inventory (first 400 chars)
[{"available_units":20,"blood_group":"A+","expired_units":0,"hospital_id":"HOSP-
001","low_stock_threshold":10,"reserved_units":0,"updated_at":"Mon, 20 Apr 2026 12:00:00 GMT"},
{"available_units":15,"blood_group":"A-","expired_units":0,"hospital_id":"HOSP-
001","low_stock_threshold":10,"reserved_units":0,"updated_at":"Mon, 20 Apr 2026 12:00:00 GMT"},{"availab

GET /api/summary/latest
[{"blood_group":"A+","summary_date":"Mon, 20 Apr 2026 00:00:00
GMT","total_units_available":324,"total_units_donated":750,"total_units_expired":0,"total_units_requested":426},{"blood_group":"A-
","summary_date":"Mon, 20 Apr 2026 00:00:00
GMT","total_units_available":120,"total_units_donated":300,"total_units_expired":0,"total_units_requested":180}]

ishan@Ubuntu:~/Blood-Bank-Inventory-Tracker$
```

REST API Health & Inventory Verification



Streamlit Live Inventory Dashboard

5. Technology Stack

- **Programming & Automation:** Python 3.12 used for core pipeline logic, data generation, processing scripts, and backend development.
- **Real-Time Data Streaming:** Apache Kafka with kafka-python for publishing and consuming donation/request events in real time.
- **Data Processing & Analytics:** PySpark for distributed batch analytics and Pandas as a fallback for local processing.
- **Database & Storage:** PostgreSQL 17 for managing inventory, transaction logs, and daily summary records.
- **API & Visualization:** Flask REST API for data access, and Streamlit dashboard for KPIs, alerts, charts, and monitoring.
- **Configuration & Environment:** python-dotenv for environment management, with Windows/Linux support and Java 17+ for Kafka and Spark execution.

6. Unique Points of the Project

- **Complete Real-Time Data Engineering Pipeline:** The project integrates data generation, Kafka streaming, PostgreSQL storage, batch analytics, REST API services, and dashboard visualization into one end-to-end healthcare solution instead of isolated modules.
- **Reliable Hybrid Processing Architecture:** It combines real-time event processing with Apache Kafka and daily analytics using PySpark, while also

providing an automatic Pandas fallback to ensure uninterrupted operation when Spark is unavailable.

- **Intelligent Inventory Management:** The system not only tracks stock levels but also detects shortages, supports partial fulfilment of requests, monitors expiry units, and triggers low-stock alerts for faster emergency response.
- **Actionable Decision-Making Dashboard:** A live Streamlit dashboard transforms backend data into usable insights through KPIs, charts, alerts, and inventory tables, making blood bank operations more efficient and transparent.

7. Future Improvements

- **AI-Based Demand Forecasting:** Integrate machine learning models to predict future blood demand based on seasonal trends, emergencies, and historical donation/request patterns.
- **Multi-City Cloud Deployment:** Deploy the system on cloud platforms to connect multiple cities, hospitals, and regional blood banks through a centralized scalable infrastructure.
- **Mobile & Notification Integration:** Develop mobile applications and automated SMS/email alerts for donors, hospitals, and administrators regarding shortages, donation drives, and urgent requests.
- **Enhanced Security & Compliance:** Implement role-based access control, data encryption, audit monitoring, and healthcare compliance standards for secure handling of sensitive records.

8. Conclusion

The Blood Bank Inventory Tracker demonstrates how modern data engineering technologies can transform traditional healthcare inventory systems into an intelligent, scalable, and real-time solution. By integrating data simulation, Kafka-based streaming, PostgreSQL storage, PySpark analytics, REST APIs, and an interactive Streamlit dashboard, the project ensures accurate stock monitoring, faster shortage detection, reduced wastage, and efficient emergency response. Its automated pipeline improves operational transparency and supports data-driven decision-making for hospitals and blood banks. The project not only addresses current challenges in blood supply management but also provides a strong foundation for future enhancements such as AI forecasting, cloud deployment, and mobile integration, making it a practical and impactful healthcare technology solution.